

YERUKULA RAGHAVENDRA

Senior Python Backend Developer

+91-6362113711 | raghavendra949072@gmail.com | Hyderabad, India

PROFESSIONAL SUMMARY

Senior Python Backend Developer with 5.6+ years of expertise in designing and developing scalable web applications, REST APIs, and ETL pipelines. Proficient in Python, Django, Django REST Framework (DRF), and FastAPI. Proven track record in AWS S3 integration, asynchronous processing, ORM tools (SQLAlchemy), and Agile/Scrum methodologies. Adept at leading technical initiatives, mentoring peers, and delivering production-grade backend systems with robust logging, error handling, and performance optimization across diverse domains including fintech, e-commerce, and HR tech.

TECHNICAL SKILLS

Languages	Python, SQL
Frameworks	Django, Django REST Framework (DRF), FastAPI
Libraries	NumPy, Pandas, aioboto3
Databases	MySQL, PostgreSQL
Cloud & Storage	AWS S3 (aioboto3), S3-compatible storage
Frontend	HTML, CSS, JavaScript, Bootstrap
Version Control	Git, GitHub, GitLab
Tools & IDEs	PyCharm, VS Code, SonarQube
Methodologies	Agile

WORK EXPERIENCE

Senior Software Engineer (Delv) | Mphasis Limited

August 2025 – Present • Hyderabad

Senior Software Engineer | Tech Mahindra Pvt. Ltd.

September 2024 – July 2025 • Bangalore

Software Engineer II | Tessolve Semiconductor Pvt. Ltd.

June 2023 – July 2024 • Bangalore

Software Engineer | Antrix Technology India Pvt. Ltd.

April 2020 – June 2023 • Bangalore

KEY PROJECTS

CANVAS – Scalable Data Ingestion & Processing Platform

Tech Stack: Python, FastAPI, AWS S3, aioboto3, PostgreSQL, SonarQube, Git

A scalable ETL-based platform to ingest, validate, transform, and persist contract and product data from multiple upstream sources into centralized entities, supporting daily reconciliation workflows and downstream reporting.

- Architected an end-to-end asynchronous Python ETL pipeline using FastAPI and aioboto3 to ingest large-scale contract and document data from S3-compatible storage, reducing processing latency by leveraging concurrent I/O operations.

- Designed a modular ETL framework with clearly separated ingestion, validation, transformation, and persistence layers, improving code maintainability and enabling independent scaling of each stage.
- Implemented date-driven incremental data processing using ETL control tables, enabling reliable daily runs, seamless recovery from failures, and zero data duplication across reprocessing scenarios.
- Built a recursive JSON flattening engine to convert deeply nested hierarchical contract document structures into normalized, relational database-friendly schemas for downstream consumption.
- Integrated AWS S3 via aioboto3 for paginated, parallel object listing and retrieval, optimizing throughput for high-volume file processing across multiple S3 buckets.
- Implemented configurable batch processing with dynamic chunk sizing to maximize memory efficiency and throughput when handling millions of records per ETL run.
- Performed systematic performance profiling of ETL stages, identifying and resolving bottlenecks that reduced total pipeline execution time by optimizing database write patterns and async task scheduling.
- Introduced connection pooling and query optimization strategies for bulk upsert operations, reducing database round trips and improving overall pipeline throughput.
- Enforced production-grade code quality standards using SonarQube for static analysis, achieving significant reduction in code smells, vulnerabilities, and duplications across the codebase.
- Implemented structured JSON logging with correlation IDs across all pipeline stages, enabling end-to-end traceability and faster root-cause analysis during incidents.
- Built retry-safe execution mechanisms with idempotent processing logic to ensure system stability and data integrity during partial failures and ETL re-runs.
- Designed and maintained ETL audit tables (ETL_driver, ETL_stats) capturing file counts, document counts, execution duration, duplicate detection, and status for operational monitoring and SLA reporting.
- Collaborated cross-functionally with data engineering, product, and downstream teams to define and align ETL data contracts, API schemas, and entitlement processing requirements.
- Participated in architecture review sessions and contributed to technical design documents, helping the team adopt best practices for scalable data platform development.

Employee Engagement Platform

Tech Stack: Python, Django, DRF, MySQL, NumPy, HTML, CSS, Bootstrap, Git

A campaign management platform enabling organizations to award points, run anniversary/nomination campaigns, and distribute reward vouchers to employees for enhanced engagement.

- Designed and developed RESTful APIs using Django REST Framework to support campaign lifecycle management including creation, participation tracking, point allocation, and voucher redemption workflows.
- Architected a flexible reward engine supporting multiple campaign types (anniversary, nominations, e-cards, group campaigns) with configurable point rules and expiry logic.
- Built a voucher management module handling value assignment, expiry dates, redemption scheduling, and currency-based point conversion for multi-tenant organizations.
- Developed a multi-tenant data model enabling isolated campaign and reward configurations per organization while sharing underlying infrastructure efficiently.
- Implemented inventory management system to track product quantities, stock thresholds, and availability across reward catalog items in real time.
- Optimized complex ORM queries for leaderboard and campaign analytics endpoints, reducing average API response time through strategic use of select_related, prefetch_related, and database indexing.
- Introduced pagination and filtering on high-traffic list endpoints to prevent N+1 query issues and improve scalability under concurrent user load.
- Implemented role-based access control (RBAC) to enforce permissions at the API level, ensuring tenants could only access and modify their own campaign and employee data.
- Wrote comprehensive unit and integration test suites using Django's TestCase and mock frameworks, achieving high test coverage across critical business logic modules.
- Conducted thorough peer code reviews focused on code readability, DRY principles, and security best practices, driving measurable improvements in overall code quality.

- Actively participated in daily standups and two-week sprint ceremonies, providing accurate effort estimates and raising blockers early to maintain sprint velocity.
- Collaborated closely with front-end developers to define API contracts, ensuring seamless integration between backend services and UI components.
- Contributed to sprint retrospectives by identifying process improvements that streamlined the team's development workflow and reduced regression defects.

E-Commerce Portal Development

Tech Stack: Python, Django, DRF, MySQL, NumPy, HTML, CSS, Bootstrap, Git

An e-commerce solution for Carrefour Indonesia supporting online product browsing, order placement, online payments, home delivery/store pickup, barcode scanning, and ERP integration for real-time stock and sales updates.

- Designed and built the full backend of an e-commerce platform using Django MVT architecture, handling product catalog, cart management, order processing, and payment integration workflows.
- Developed RESTful APIs for product search, order placement, payment processing, and delivery-mode selection (home delivery vs. store pickup), ensuring seamless mobile and web client integration.
- Built a real-time ERP integration layer to synchronize product master data, live stock levels, and daily sales figures between the e-commerce platform and Carrefour's ERP system.
- Implemented barcode scanning functionality integrated with the product catalog, enabling rapid item lookup and cart addition for in-store and online use cases.
- Utilized OOP principles including class-based views, mixins, and model inheritance to build reusable, extensible backend components with high cohesion and low coupling.
- Implemented caching strategies for product catalog and stock data using Django's caching framework, significantly reducing database load during peak traffic periods.
- Optimized database schema with appropriate indexing on high-frequency query fields (product SKU, category, order status), improving query performance for catalog and order history endpoints.
- Applied input validation, SQL injection prevention via ORM parameterized queries, and secure session management to protect customer data and payment flows.
- Integrated HTTPS enforcement and token-based authentication for all API endpoints, ensuring compliance with e-commerce security standards.
- Maintained high code quality through active participation in peer-to-peer code review cycles, identifying and resolving potential security vulnerabilities and logic errors pre-deployment.
- Worked closely with the QA team through structured test cycles, triaging defects, providing RCA (root cause analysis), and deploying timely fixes to maintain release schedules.
- Followed Agile/Scrum SDLC with test-driven development, consistently delivering sprint goals and maintaining documentation for API contracts and data flow diagrams.

Test Framework for Web Services (TWS)

Tech Stack: Python, Django, REST API, MySQL, Git

A versatile testing and automation framework for OS and platform validation, supporting software and driver development with minimal coding via command-line interfaces.

- Designed the core architecture of the TWS framework with a plugin-based, modular structure enabling extensibility across different OS platforms, hardware configurations, and test scenario types.
- Implemented a CLI-driven test orchestration engine that abstracts test creation complexity, allowing engineers to define and run test suites with minimal boilerplate code.
- Developed parallelization infrastructure enabling simultaneous execution of test suites across multiple physical and virtual systems, dramatically reducing total validation cycle time.
- Built a parameterization engine supporting dynamic test configuration, enabling targeted hardware-specific testing through runtime variable injection without code modification.
- Implemented multi-language script integration layer supporting Python, shell, and other scripting languages, broadening the framework's applicability across diverse test automation needs.
- Optimized test scheduler to maximize resource utilization during parallel test runs, implementing load balancing across available test nodes to minimize idle time and bottlenecks.

- Introduced incremental test execution support, allowing partial re-runs of only failed or modified test cases, reducing feedback cycle time during iterative driver and firmware development.
- Implemented structured logging and centralized test result storage in MySQL, enabling historical trend analysis, failure pattern detection, and audit trail for compliance requirements.
- Enforced code quality standards across the framework codebase through rigorous code reviews and adherence to PEP 8 and internal Python coding guidelines.
- Developed real-time visual diagnostic dashboards with screenshot capture on test failures, giving developers immediate visual context for debugging hardware and driver issues.
- Collaborated with hardware, firmware, and software engineering teams to define test coverage requirements and ensure the framework addressed real-world validation scenarios effectively.
- Documented framework architecture, CLI usage, and contribution guidelines, accelerating onboarding for new team members and promoting consistent usage across multiple engineering teams.

EDUCATION

Bachelor of Technology – Electrical and Electronics Engineering

Bheema Institute of Technology & Science, Adoni | JNTU Anantapur University | 2018